

JavaAgent-CLI

开发和安装手册

基于 ReAct 架构的自主编程智能体

版本：1.0.0

日期：2026 年 6 月 12 日

开发者（排名不分先后）：莫承潜，黄春云，胡鸿扬

汇报者（排名不分先后）：黄麟松，王郅为

四川农业大学 · 信息工程学院

仓库地址：<https://github.com/cuac333/JavaAgent-CLI>

本项目采用 **MIT 开源协议** (MIT License) 发布。

版权所有 © 2026 四川农业大学信息工程学院 JavaAgent-CLI 开发团队。

在遵守 MIT 协议条款的前提下，任何人都可以自由使用、复制、修改、合并、发布、分发、再许可和/或出售本软件的副本。

目录

一、项目简介

二、系统架构

三、环境与安装

四、配置说明

五、使用指南

六、安全机制

七、架构与项目结构

八、常见问题

九、开发与实现

9.1 Step 1: 搭建项目骨架 + 最简 ReAct 循环

9.2 Step 2: 工具系统 + 用户审批机制

9.3 Step 3: 界面美化 + 整体验证

9.4 团队分工与协作说明

一、项目简介

1.1 什么是 JavaAgent-CLI

JavaAgent-CLI 是一个用 Java 21 实现的自主编程智能体，类似于 Claude Code 的命令行版本。它采用 **ReAct (Reasoning + Acting) 架构**，让大语言模型通过工具调用与本地开发环境交互，自动完成编程任务。

用户输入自然语言指令后，智能体自主规划、推理、调用工具（文件编辑、搜索、Shell 执行等）来完成任务，整个过程在终端中实时可见。

1.2 核心特性

特性	说明
ReAct 推理循环	观察-推理-行动-反思，每轮最多 12 次迭代

8 个内置工具	read_file、write_file、edit、delete_file、grep、list_directory、bash、network
SSE 流式输出	逐 token 实时显示 AI 回复，支持思维链推理
人机协作审批	只读操作自动批准，写入操作需用户确认
上下文可视化	/context 命令展示 token 使用分布
5 级推理深度	从"简短直接"到"极限推理"可调
安全沙箱	工作区隔离、危险命令检测、敏感信息脱敏
跨平台支持	macOS / Linux / Windows（PowerShell/cmd）

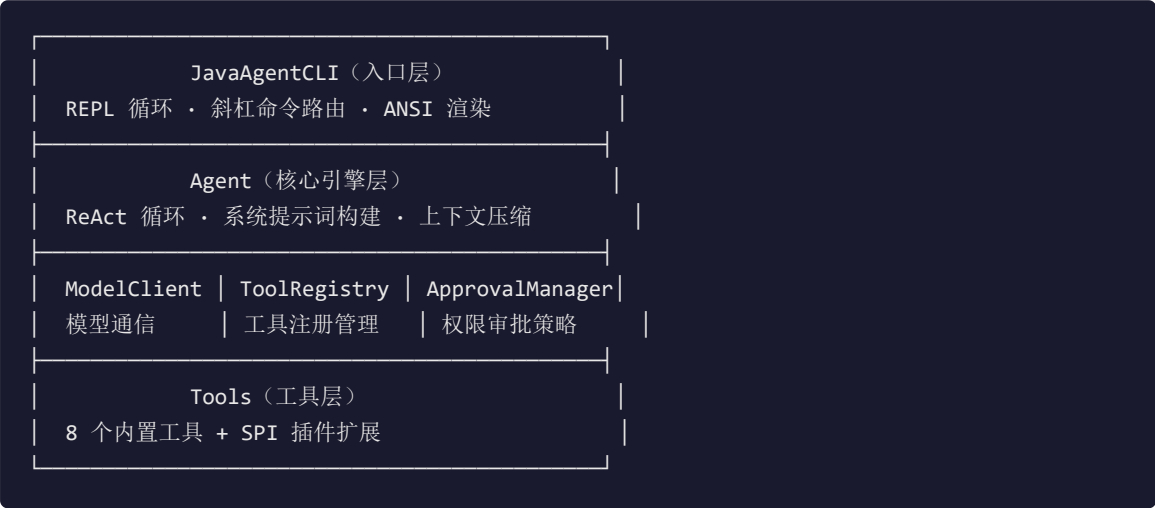
1.3 技术栈

组件	版本	用途
Java	21+	运行环境
Maven	3.8+	构建工具
Jackson	2.18.3	JSON 序列化
JLine 3	3.28.0	终端交互、行编辑、Tab 补全
jtokkit	1.1.0	Token 计数（cl100k_base）
JUnit 5	5.11.4	单元测试与集成测试

二、系统架构

2.1 整体架构

JavaAgent-CLI 采用分层架构设计，各层职责清晰：



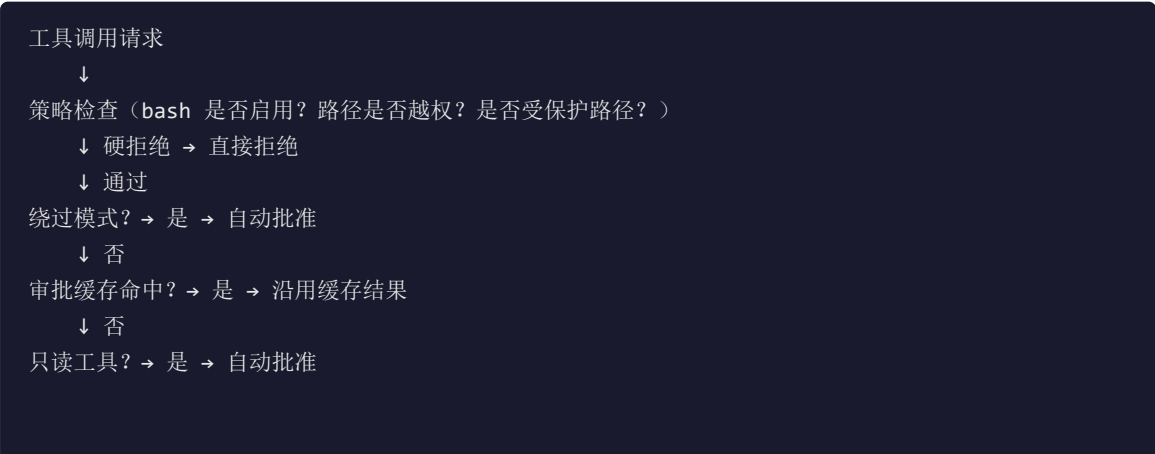
2.2 ReAct 循环流程

每次用户输入后，Agent 执行以下循环：

1. 将用户消息加入对话历史
2. 构建系统提示词（身份 + 规则 + 工具定义）
3. 调用 LLM 获取响应（文本回复 或 工具调用）
4. 如果响应包含工具调用 → 执行工具 → 将结果返回 LLM → 回到第 3 步
5. 如果响应是文本回复 → 返回给用户
6. 达到最大迭代次数（默认 12）→ 停止并提示

2.3 审批流程

工具执行前经过审批检查：



↓ 否

请求用户确认 → 缓存结果 → 执行/拒绝

三、环境与安装

3.1 所需环境

软件	最低版本	说明
JDK	21	需要支持虚拟线程和模式切换
Maven	3.8	构建工具
操作系统	—	macOS、Linux、Windows 均可

3.2 获取 API Key

OpenAI 官方

1. 访问 <https://platform.openai.com> 注册账号
2. 进入 Settings → API Keys → Create new secret key
3. 复制生成的 Key（格式：sk-...），仅显示一次
4. 记录 Base URL：<https://api.openai.com/v1>

第三方兼容服务

DeepSeek、Mimo 等平台流程类似：注册 → 控制台创建 API Key → 记录 Base URL。

提示：JavaAgent CLI 使用标准 OpenAI Chat Completions 接口，任何兼容该接口的服务都可以直接使用。无需 API Key 可用 Mock 模式（`--mock`）。

3.3 获取源码

```
git clone https://github.com/cuac333/JavaAgent-CLI.git
cd JavaAgent-CLI
```

3.4 构建项目

```
mvn clean package -DskipTests
```

成功后会在 `target/` 目录下生成 `javaagent-cli-1.0.0.jar`（约 15MB 的 fat JAR）。

3.5 配置 API Key

在项目根目录创建 `config.properties` 文件：

```
agent.mock_mode=false
agent.api_key=sk-your-api-key-here
```

```
agent.base_url=https://api.openai.com/v1
agent.model=gpt-5.4-mini
```

3.6 运行程序

macOS / Linux

```
# 方式一：使用包装脚本（推荐）
chmod +x javaagentcli
./javaagentcli

# 方式二：直接运行 JAR
java --enable-native-access=ALL-UNNAMED -jar target/javaagent-cli-1.0.0.jar
```

Windows

```
# 方式一：使用 cmd 包装脚本
javaagentcli.cmd

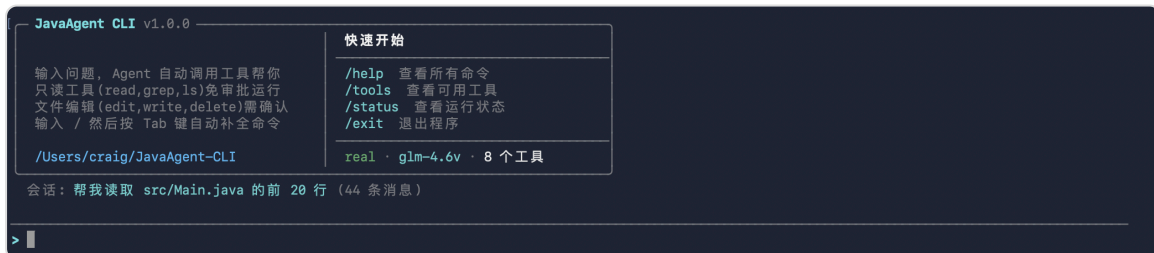
# 方式二：直接运行 JAR
java --enable-native-access=ALL-UNNAMED -jar target\javaagent-cli-1.0.0.jar
```

模拟模式（无需 API Key）

```
java --enable-native-access=ALL-UNNAMED -jar target/javaagent-cli-1.0.0.jar --mock
```

3.7 验证运行

启动成功后将看到如下界面：



在 `>` 提示符处输入 `hello`，如果 Agent 回复了一段文字，说明环境配置成功。

审批操作：需要确认的工具会显示 `允许执行? [Y/n]`，直接按 **Enter** 或输入 `y` + Enter 确认，输入 `n` + Enter 拒绝。

四、配置说明

运行时行为通过 `config.properties` 文件控制。程序会按以下顺序查找配置文件：

1. 项目根目录（与 JAR 同级）
2. 当前工作目录
3. `~/.javaagent-cli/config.properties`

4.1 完整配置项

```
# ===== 基础配置 =====
agent.mock_mode=false           # 模拟模式开关
agent.api_key=                   # OpenAI 兼容 API Key
agent.base_url=https://api.openai.com/v1 # API 基础地址
agent.model=gpt-5.4-mini        # 模型名称

# ===== 行为配置 =====
agent.max_iterations=12         # 每轮最大工具调用次数（1-50）
agent.enable_bash=false        # 是否启用 bash 工具（默认禁用）
agent.stream_responses=true     # 是否启用流式输出
agent.effort=high               # 推理深度（low/high/xhigh/max/ultra）
agent.auto_save=true           # 自动保存会话
agent.approval_cache=true      # 审批缓存开关
agent.allow_external_paths=false # 是否允许访问工作区外路径
agent.bypass_permissions=false  # 跳过所有审批（危险！）

# ===== 上下文配置 =====
agent.system_prompt=           # 自定义 System Prompt（留空使用默认）
agent.max_tokens=200000        # 上下文窗口大小
agent.compact_threshold=0.8    # 自动压缩阈值（0.0-1.0）
agent.rate_limit_qps=10        # API 速率限制（QPS）
```

4.2 配置示例

```
# 模拟模式（开发/演示用）
agent.mock_mode=true

# 真实模式（以 DeepSeek 为例）
agent.mock_mode=false
agent.api_key=sk-your-api-key-here
agent.base_url=https://api.deepseek.com/v1
agent.model=deepseek-chat
agent.enable_bash=false
```

提示：配置修改后可在 CLI 中使用 `/reload` 命令热重载，无需重启程序。

五、使用指南

5.1 基本交互

启动程序后，在提示符处输入自然语言指令即可：

```
> 帮我读取 src/Main.java 的前 20 行
> 搜索项目中所有 TODO 注释
> 把 Foo.java 里的 oldMethod 改名为 newMethod
```

输入 `/` 然后按 **Tab** 键可以自动补全命令，上下键浏览历史输入。

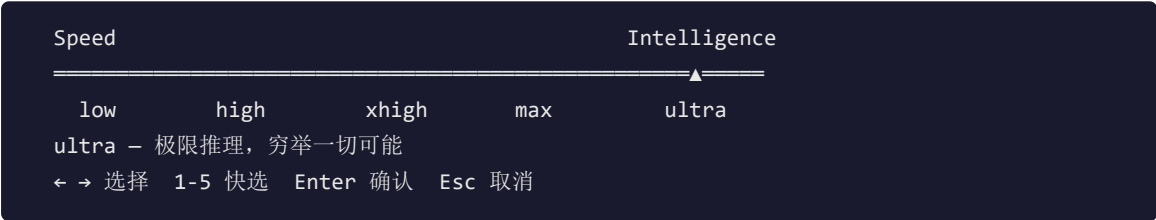
5.2 斜杠命令

命令	说明
<code>/help</code>	显示帮助信息
<code>/exit</code> <code>/quit</code>	退出程序
<code>/clear</code> <code>/new [title]</code>	新建会话
<code>/save [title]</code>	保存当前会话
<code>/load [id title latest]</code>	加载已保存的会话
<code>/sessions</code>	列出已保存的会话
<code>/tools</code>	列出已注册的工具
<code>/mode mock real</code>	切换模型模式
<code>/stream on off</code>	开关流式输出
<code>/bash on off</code>	开关 Bash 工具
<code>/bypass on off</code>	跳过所有工具审批确认
<code>/effort [level]</code>	调节推理深度 (low/high/xhigh/max/ultra)
<code>/context</code>	查看上下文 token 占用
<code>/compact</code>	压缩对话历史
<code>/prompt show set reset</code>	管理自定义 System Prompt
<code>/approvals clear</code>	清空审批缓存
<code>/reload</code>	重新加载配置文件
<code>/export</code>	导出当前对话为 Markdown
<code>/stats</code>	查看工具调用统计

5.3 推理深度

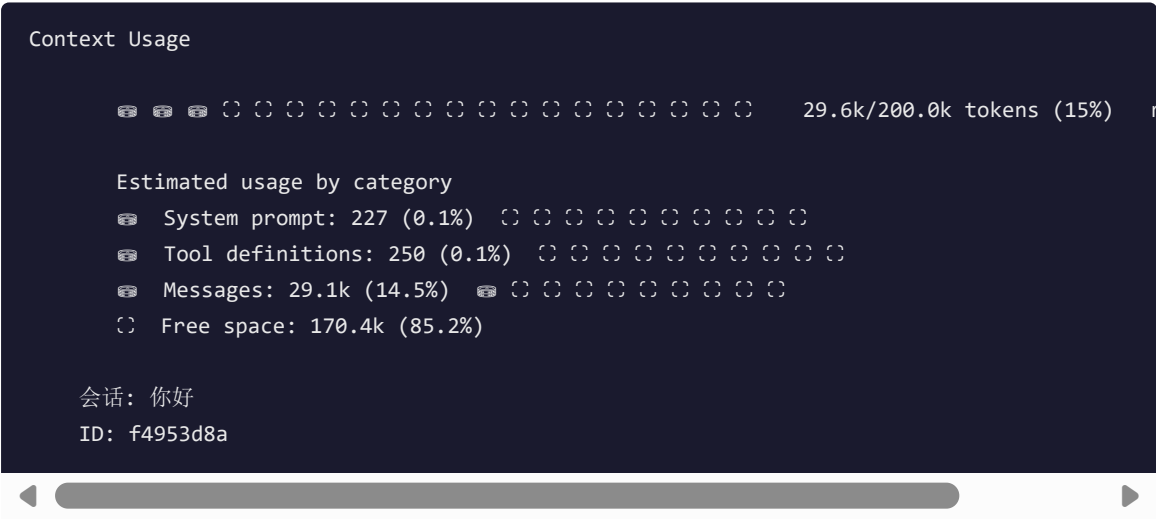
级别	特点	适用场景
low	简短直接，跳过解释	快速问答、简单操作
high	逐步推理，详细解释	日常编程任务（默认）
xhigh	深度分析，多角度探索	复杂问题、架构设计
max	最大深度，考虑所有边界	关键代码审查
ultra	极限推理，穷举一切可能	极其复杂的系统设计

使用 `/effort` 不带参数时，会弹出交互式滑块选择器：



5.4 上下文可视化

使用 `/context` 命令查看当前 token 使用情况，显示彩色柱状图：



当 token 使用超过阈值（默认 80%）时，系统会自动压缩对话历史。也可使用 `/compact` 手动压缩。

5.5 内置工具

工具名	别名	功能	权限
<code>read_file</code>	cat	读取文件内容（256KB 上限，自动分页）	只读

<code>write_file</code>	write, save_file	写入文件（100KB 上限，60 行预览）	读写
<code>edit</code>	replace, sed	精确字符串替换（带 diff 预览）	读写
<code>delete_file</code>	rm	删除文件（禁止删除工作区根目录）	读写
<code>grep</code>	search, find	正则搜索（跳过 target/.git，最多 100 匹配）	只读
<code>list_directory</code>	ls, dir	列出目录内容	只读
<code>bash</code>	shell, exec	执行 Shell 命令（默认禁用，10s 超时）	读写
<code>network</code>	http, fetch	发送 HTTP 请求	读写

工具执行显示效果

只读工具自动执行，写操作需用户确认。以下是典型显示：

list_directory（列目录） — `[D]` 目录，`[F]` 文件：

```
└─ list_directory - .
| 执行工具中...
| ✓ 完成
| recursive=false
| [D] src
| [D] target
| [F] .gitignore
| [F] pom.xml
| entries=25
```

read_file（读文件） — 显示行号和内容：

```
└─ read_file - test_tools.txt
| 执行工具中...
| ✓ 完成
| Lines: 1-6 of 6
|   1 | 这是一个测试文件。
|   2 | 第一行内容。
| ... (还有 4 行)
```

edit（编辑） — 需确认，显示 diff 预览：

```
└─ edit - test_tools.txt
| 执行工具中...

允许执行? [Y/n]
edit → test_tools.txt
preview:
| - 第一行内容。
| + 第一行已修改。
y
✓ 已允许
| ✓ 完成
```

审批操作：直接按 **Enter** 或输入 **y** + Enter 确认，输入 **n** + Enter 拒绝。首次批准后，后续相同操作自动放行（可通过 **/approvals clear** 清空缓存）。

六、安全机制

6.1 工作区隔离

文件操作默认限制在工作区内。访问工作区外的路径会被拒绝，除非在配置中明确启用 `agent.allow_external_paths=true`。

6.2 危险命令检测

BashTool 内置了 20 种危险命令模式检测：

类别	检测模式
文件系统破坏	rm -rf /、rm -rf ~、chmod -R 777 /
磁盘破坏	mkfs、dd if=...of=/dev/
系统破坏	shutdown、:(){: :& }::
安全攻击	反弹 shell、curl bash、kill -9 1
数据泄露	curl -d @/etc/passwd、shred

6.3 敏感信息脱敏

工具执行结果在存储到会话文件前，会自动脱敏以下内容：

- API Key（sk-xxx 格式）
- Bearer Token
- agent.api_key 配置值

6.4 连续失败熔断

同一工具连续失败 3 次后，Agent 自动停止并提示，防止无限循环。

七、架构与项目结构

7.1 插件扩展

JavaAgent-CLI 支持通过 Java SPI (ServiceLoader) 机制扩展新工具：

1. 实现 `ToolProvider` 接口
2. 在 `META-INF/services/com.javagent.tools.ToolProvider` 中注册
3. 将 JAR 放入 classpath
4. 程序启动时自动发现并注册新工具

```
public interface ToolProvider {  
    /** 返回工具定义列表 */  
    List<Tool> provideTools();  
}
```

7.2 项目结构

```
JavaAgent-CLI/  
├── src/main/java/com/javagent/  
│   ├── JavaAgentCLI.java          # CLI 入口, REPL 循环  
│   ├── BannerPrinter.java         # 启动横幅 + 状态栏  
│   ├── SlashCommandCompleter.java # 斜杠命令补全  
│   ├── core/  
│   │   ├── Agent.java            # ReAct 循环引擎  
│   │   ├── Config.java           # 配置管理  
│   │   ├── ConversationManager.java # 会话管理  
│   │   ├── ApprovalManager.java  # 权限审批  
│   │   └── ...  
│   ├── model/  
│   │   ├── ModelClient.java      # 模型客户端接口  
│   │   ├── OpenAiCompatibleModelClient.java  
│   │   ├── MockModelClient.java  # 模拟客户端  
│   │   └── ...  
│   ├── tools/  
│   │   ├── Tool.java             # 工具接口  
│   │   ├── ToolRegistry.java     # 工具注册中心  
│   │   ├── ReadFileTool.java     # 读取文件  
│   │   ├── EditTool.java         # 精确编辑  
│   │   ├── BashTool.java         # Shell 执行  
│   │   └── ...  
│   └── util/  
│       ├── Terminal.java         # ANSI 终端渲染  
│       ├── TokenCounter.java     # Token 计数  
│       ├── MarkdownRenderer.java # Markdown 渲染  
│       └── ...  
└── src/test/java/                # 66 个测试用例
```

```
|— pom.xml                # Maven 配置
|— javaagentcli           # macOS/Linux 启动脚本
|— javaagentcli.cmd       # Windows 启动脚本
|— config.properties     # 配置文件（可选）
```

八、常见问题

Q1: 启动时报 "Module enable-native-access" 错误

需要添加 JVM 参数: `--enable-native-access=ALL-UNNAMED`。使用提供的包装脚本可自动添加此参数。

Q2: 如何切换 AI 模型?

修改 `config.properties` 中的 `agent.model` 和 `agent.base_url`, 然后在 CLI 中执行 `/reload`

Q3: bash 工具默认禁用, 如何开启?

在 `config.properties` 中设置:
`agent.enable_bash=true`

或在 CLI 中使用 `/bash on` 命令开启。

Q4: 如何查看 token 使用情况?

输入 `/context` 命令, 会显示彩色的 token 使用分布图, 包括系统提示词、工具定义、对话消息的分别占用。

Q5: 会话数据保存在哪里?

`~/.javaagent-cli/sessions/`

格式为 JSON。

Q6: 如何自定义系统提示词?

```
/prompt set <文本>    # 设置自定义提示词
/prompt show          # 查看当前提示词
/prompt reset         # 恢复默认
```

Q7: API 连接报错怎么办?

错误	原因	解决方案
----	----	------

Connection refused	网络不通或 base_url 错误	检查网络、确认 URL 末尾有 /v1
401 Unauthorized	API Key 无效/过期	检查 Key 无空格、在有效期内
429 Rate limit	请求过于频繁	降低 agent.rate_limit_qps=5
404 Model not found	model 名称错误	确认模型名与平台一致

Q8: Windows 上中文乱码怎么办？

使用 Windows Terminal，或在 cmd.exe 中先执行 `chcp 65001` 切换到 UTF-8 编码。

九、开发与实现

从零搭建一个 ReAct 编程智能体 —— 3 个 Step 渐进式开发

本章按照**渐进式开发**的思路，分 3 个 Step 带你从零实现一个 ReAct 编程智能体。每个 Step 都是一个可独立运行的里程碑。

9.1 Step 1: 搭建项目骨架 + 实现最简 ReAct 循环

目标

让 Agent 能调用一次 LLM、解析输出、决定是否调用工具。先不做真实工具，只打印日志。

具体做了什么

1. **创建 Maven 项目**：配置 `pom.xml`，引入 Jackson（JSON 解析）和 JLine3（终端交互）依赖
2. **封装 OpenAI 兼容客户端**：用 `java.net.http.HttpClient` 发送 POST 请求到 `/v1/chat/completions`，解析 SSE 流式响应
3. **定义消息模型**：用 Java 21 Record 定义 `Message`、`ToolCall`、`ModelResponse`
4. **实现 ReAct 主循环**：发送对话 → 解析响应 → 如果有 `tool_calls` 就记录日志 → 循环直到返回纯文本

关键代码片段

模型客户端核心调用（简化版）：

```
public ModelResponse chat(List<Message> messages, List<ToolDefinition> tools) {
    // 1. 构建请求 JSON
    Map<String, Object> body = Map.of(
        "model", config.model(),
        "messages", messages,
        "tools", tools,
        "stream", true
    );
    // 2. 发送 HTTP POST
    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(config.baseUrl() + "/chat/completions"))
        .header("Authorization", "Bearer " + config.apiKey())
        .POST(HttpRequest.BodyPublishers.ofString(
            objectMapper.writeValueAsString(body)))
        .build();
    // 3. 发送并解析 SSE 流式响应
    HttpResponse<InputStream> response = httpClient.send(
        request, HttpResponse.BodyHandlers.ofInputStream());
}
```

```
        return parseSSE(response.body());
    }
}
```

ReAct 主循环（简化版）：

```
while (iterations < maxIterations) {
    ModelResponse response = modelClient.chat(conversation, tools);
    if (response.hasToolCalls()) {
        // 记录工具调用日志（暂不执行）
        for (ToolCall call : response.toolCalls()) {
            System.out.println(" Thought: 需要调用 " + call.name());
            System.out.println(" Action: " + call.name()
                               + "(" + call.arguments() + ")");
        }
        conversation.add(assistantMessage);
    } else {
        // 纯文本回复，结束循环
        System.out.println(response.text());
        break;
    }
    iterations++;
}
```

遇到的坑

问题	原因	解决方案
SSE 解析乱码	流式响应按 chunk 返回，需要逐行解析 <code>data:</code> 前缀	用 <code>BufferedReader</code> 逐行读取，跳过 <code>[DONE]</code> 标记
LLM 返回格式不稳定	不同模型的 <code>tool_calls</code> JSON 结构略有差异	加容错：检查 <code>function</code> 字段是否存在，缺失时尝试从 <code>arguments</code> 字符串解析
对话历史越来越长	每轮都追加消息，token 快速耗尽	预留 <code>compact()</code> 接口，Step 3 再实现

验证

```
> 今天天气怎么样
Thought: 需要调用 weather 工具
Action: weather({"city": "北京"})
Thought: 需要调用 weather 工具
Action: weather({"city": "北京"})
... (循环直到达到最大次数)
```

看到 Agent 打印出 Thought 和 Action，说明 ReAct 循环工作正常。

如果你想自己实现：核心就是 "发送对话 → 解析响应 → 判断是否有 tool_calls → 循环" 这四步。先用假数据跑通流程，再接入真实工具。

9.2 Step 2: 实现工具系统 + 用户审批机制

目标

让 Agent 真正能执行 Bash 命令、读写文件，并在危险操作前请求用户确认。

具体做了什么

1. 定义 Tool 接口：统一的 `execute(Map<String, String>)` 方法，返回执行结果字符串
2. 实现 8 个工具：ReadFileTool、WriteFileTool、EditTool、DeleteFileTool、GrepTool、ListDirectoryTool、BashTool、NetworkTool
3. 实现 ToolRegistry：按名称和别名查找工具，支持 SPI 插件扩展
4. 实现 ApprovalManager：策略链审批（只读自动放行、写操作需确认、危险命令拒绝）
5. 在 ReAct 循环中接入：收到 tool_calls → 查找工具 → 审批 → 执行 → 结果回灌对话

关键代码片段

Tool 接口定义：

```
public interface Tool {  
    ToolDefinition definition();           // 工具名称、描述、参数 schema  
    String execute(Map<String, String> parameters); // 执行逻辑  
    boolean requiresApproval();           // 是否需要用户确认  
}
```

BashTool 核心执行逻辑（简化版）：

```
public String execute(Map<String, String> parameters) {  
    String command = parameters.get("command");  
    // 1. 危险命令检测（20+ 种正则模式）  
    if (isDangerous(command)) {  
        return "ERROR: 检测到危险命令，已拒绝执行";  
    }  
    // 2. 根据操作系统选择 Shell  
    String[] cmd = IS_WINDOWS  
        ? new String[]{"powershell", "-Command", command}  
        : new String[]{"bin/bash", "-c", command};  
    // 3. 执行并捕获输出  
    Process proc = Runtime.getRuntime().exec(cmd);  
    String output = new String(proc.getInputStream().readAllBytes());  
    int exitCode = proc.waitFor();  
    return output + "\n[exit=" + exitCode + "];"  
}
```

审批流程（简化版）：

```
public ApprovalOutcome authorize(ToolCall call) {  
    // 1. 绕过模式检查  
    if (config.bypassPermissions())
```

```
        return ApprovalOutcome.approved("bypass");
// 2. 只读工具自动放行
if (!tool.requiresApproval())
    return ApprovalOutcome.approved("auto");
// 3. 缓存命中检查
if (approvalCache.contains(call))
    return ApprovalOutcome.approved("cached");
// 4. 请求用户确认
System.out.println("  允许执行? [Y/n]");
String input = reader.readLine();
boolean approved = input.isEmpty()
    || input.equalsIgnoreCase("y");
if (approved) approvalCache.store(call);
return approved
    ? ApprovalOutcome.approved("user")
    : ApprovalOutcome.denied("user");
}
```

遇到的坑

问题	原因	解决方案
工具执行结果太长，token 爆了	读取大文件或长目录输出	ReadFileTool 加 256KB 上限和分页，GrepTool 限 100 匹配
LLM 不理解工具返回的错误	错误信息格式不统一	统一返回 <code>ERROR: ...</code> 前缀，让 LLM 容易识别
Bash 工具在 Windows 上失败	Linux 的 <code>/bin/bash</code> 不存在	检测 OS，Windows 用 PowerShell → cmd.exe 降级
同一工具反复失败导致死循环	LLM 不断重试同一个失败的工具	连续失败 3 次自动熔断

验证

```
> 帮我创建一个 Hello.java 文件，输出 "Hello World"，然后编译运行它

└─ write_file - Hello.java
└─ 执行工具中...
允许执行? [Y/n]          ← 用户输入 y 或直接 Enter
✓ 已允许
└─ ✓ 完成

└─ bash - javac Hello.java && java Hello
└─ 执行工具中...
允许执行? [Y/n]          ← 用户输入 y
✓ 已允许
└─ ✓ 完成
└─ Hello World

> 完成！我已经创建了 Hello.java 并成功运行，输出了 "Hello World"。
```

如果你想自己实现：核心是 "Tool 接口 + Registry 查找 + 审批拦截 + 结果回灌" 这条链路。先实现一个 BashTool 和 ReadFileTool 就能跑通大部分场景。

9.3 Step 3: 命令行界面美化 + 整体验证与优化

目标

改善用户体验，支持彩色输出、历史命令、思考动画、Markdown 渲染。

具体做了什么

1. **集成 JLine3**: 实现在编辑、Tab 自动补全 (斜杠命令 + 文件路径)、命令历史上下键浏览
2. **ANSI 彩色输出**: 用转义码实现标题蓝色、代码红色、成功绿色、警告黄色
3. **思考动画**: LLM 推理时显示 braille 字符旋转动画 (.: : " :.: .: .:)
4. **工具执行面板**: 用边框字符 (┌ │ └) 包裹工具输出, 显示状态 (执行中... / ✓ 完成)
5. **Markdown 渲染**: 将 LLM 返回的 Markdown 转为 ANSI 终端格式 (标题加粗、代码高亮等)
6. **上下文压缩**: 当 token 超过阈值时, 保留最近 6 条消息, 其余交给 LLM 总结

关键代码片段

JLine3 Tab 补全 (简化版):

```
LineReader reader = LineReaderBuilder.builder()
    .terminal(terminal)
    .completer(new StringsCompleter(
        "/help", "/exit", "/clear",
        "/context", "/effort", "/save", "/load", "/tools"))
    .build();
```

思考动画（简化版）：

```
String[] frames = {".: ", ":~ ", "???", " !:", "!..", ".!.", "!!:"};  
int i = 0;  
while (thinking) {  
    System.out.print("\r   " + frames[i % frames.length]  
        + " 思考中...");  
    i++;  
    Thread.sleep(100);  
}  
  
System.out.print("\r                               \r"); // 清除动画
```

上下文压缩 (简化版):

```
public void compact() {  
    // 1. 保留最近 6 条消息  
    List<Message> recent = history.subList(  
        history.size() - 6, history.size());  
    List<Message> old = history.subList(  
        0, history.size() - 6);  
    // 2. 让 LLM 总结旧消息  
    String summary = modelClient.chat(List.of(  
        new Message(MessageRole.SYSTEM, "你是一位资深的产品经理，擅长总结需求。"),  
        new Message(MessageRole.USER, "请总结以下需求，并给出改进建议。"),  
        old.toArray(new Message[0])
```

```
        Message.system("请用中文总结以下对话要点。"),
        Message.user(old.toString())
    ).text();
    // 3. 替换历史
    history.clear();
    history.add(Message.system("对话摘要: " + summary));
    history.addAll(recent);
}
```

遇到的坑

问题	原因	解决方案
JLine3 在某些终端不工作	嵌入式终端（如 IDE 内置）不支持 JNI	检测终端类型，不支持时降级为普通 Scanner
中文字符宽度计算错误	CJK 字符占 2 列，ASCII 占 1 列	Terminal.java 实现 <code>displayWidth()</code> ，按 Unicode 范围判断
流式输出和动画冲突	多线程同时写 System.out	用 <code>SynchronizedOutputStream</code> 串行化输出
Markdown 渲染不完整	LLM 返回的 Markdown 格式不标准	只渲染常见的标题、粗体、代码块、列表，其余原样输出

整体验证

用以下任务测试 Agent 的完整能力：

测试任务	预期行为	验证点
"读取 pom.xml 的前 10 行"	调用 read_file，显示文件内容	只读工具免审批
"搜索所有包含 TODO 的文件"	调用 grep，显示匹配结果	搜索结果格式正确
"创建一个 Calculator.java"	调用 write_file，需用户确认	审批流程正常
"解释一下 Agent.java 的核心逻辑"	读取文件后给出解释	多步工具编排
"编译并运行刚才创建的文件"	调用 bash，执行 javac + java	Bash 工具正常

已知局限与待改进

- **LLM 输出格式不严格**：不同模型返回的 tool_calls JSON 结构有差异，需要写容错解析器
- **上下文压缩会丢失细节**：总结可能遗漏重要信息，可考虑用向量检索辅助
- **并发工具调用**：当前是串行执行，未来可支持并行调用多个工具
- **插件生态**：SPI 机制已就绪，但社区插件较少，可鼓励贡献

如果你想自己实现：界面美化是锦上添花，核心价值在 Step 1 和 Step 2。先把 ReAct 循环和工具系统跑通，再考虑加 JLine3、彩色输出等体验优化。

9.4 团队分工与协作说明

以下是参考的团队分工方案（使用化名）：

成员	负责内容	具体工作
A 同学	Step 1 + Step 2 核心逻辑	搭建项目骨架、实现 ReAct 循环、封装 API 客户端、实现 Tool 接口和 8 个内置工具、审批机制。使用 AI 辅助编码生成基础框架。
B 同学	Step 3 界面美化	集成 JLine3 实现行编辑和 Tab 补全、ANSI 彩色输出、思考动画、工具执行面板、Markdown 渲染器。
C 同学	测试与优化	编写 66 个 JUnit 5 测试用例、跨平台测试（Windows/macOS/Linux）、性能优化、文档编写。

协作流程建议

- Step 1 完成后：**A 同学提交基础框架，B 同学基于此开发界面，C 同学开始写测试
- Step 2 完成后：**三人一起用实际任务测试，收集问题清单
- Step 3 完成后：**C 同学主导最终验收，A/B 同学修复 bug

AI 辅助开发实践

本项目大量使用 AI 辅助开发，以下是实践经验：

阶段	AI 辅助方式	效果
需求分析	让 AI 分析 ReAct 架构，生成类图和接口定义	快速确定技术方案
代码生成	描述需求让 AI 生成 Tool 接口、BashTool 等基础代码	节省 60% 编码时间
Bug 修复	粘贴错误日志让 AI 分析原因并给出修复方案	快速定位问题
文档编写	让 AI 根据代码生成伪代码和架构文档	文档质量高且省时

给读者的建议：不要直接让 AI 写完整个项目。正确的做法是：自己先理解架构 → 让 AI 生成单个模块 → 自己审查和修改 → 再让 AI 生成下一个模块。这样既能学习，又能保证代码质量。